Guideline Session – Debugging

Generative Software Engineering CS 630

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Team 6: Mihiret Ali, Ting Huang, Mansi Sawant, Hamid Ahmad

29.04.2026

Why debugging is the hardest part of AI-Assisted development ?

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

AI generates confident wrong code

LLMs generate buggy code as fast as correct code

Debugging loops without convergence

Blind fix it prompts loop. Chen et al. show > 3 turns rarely helps, but most developers don't know when to stop.

Silent regressions from AI fixes

AI fixes address the symptom. Incidents per pull request rose 23.5% with AI adoption (Google DORA 2025). Code churn doubled.

Static reasoning can't catch runtime bugs

Race conditions, I/O side-effects, and non-deterministic failures are invisible to any explanation-based approach.

What our guidelines covers

1. **Explain-Then-Fix**
   Rubber duck: explaining code line-by-line
2. **AutoSD**
   LLM-Driven Scientific Debugging
3. **RepairAgent**
   Find and fix bugs

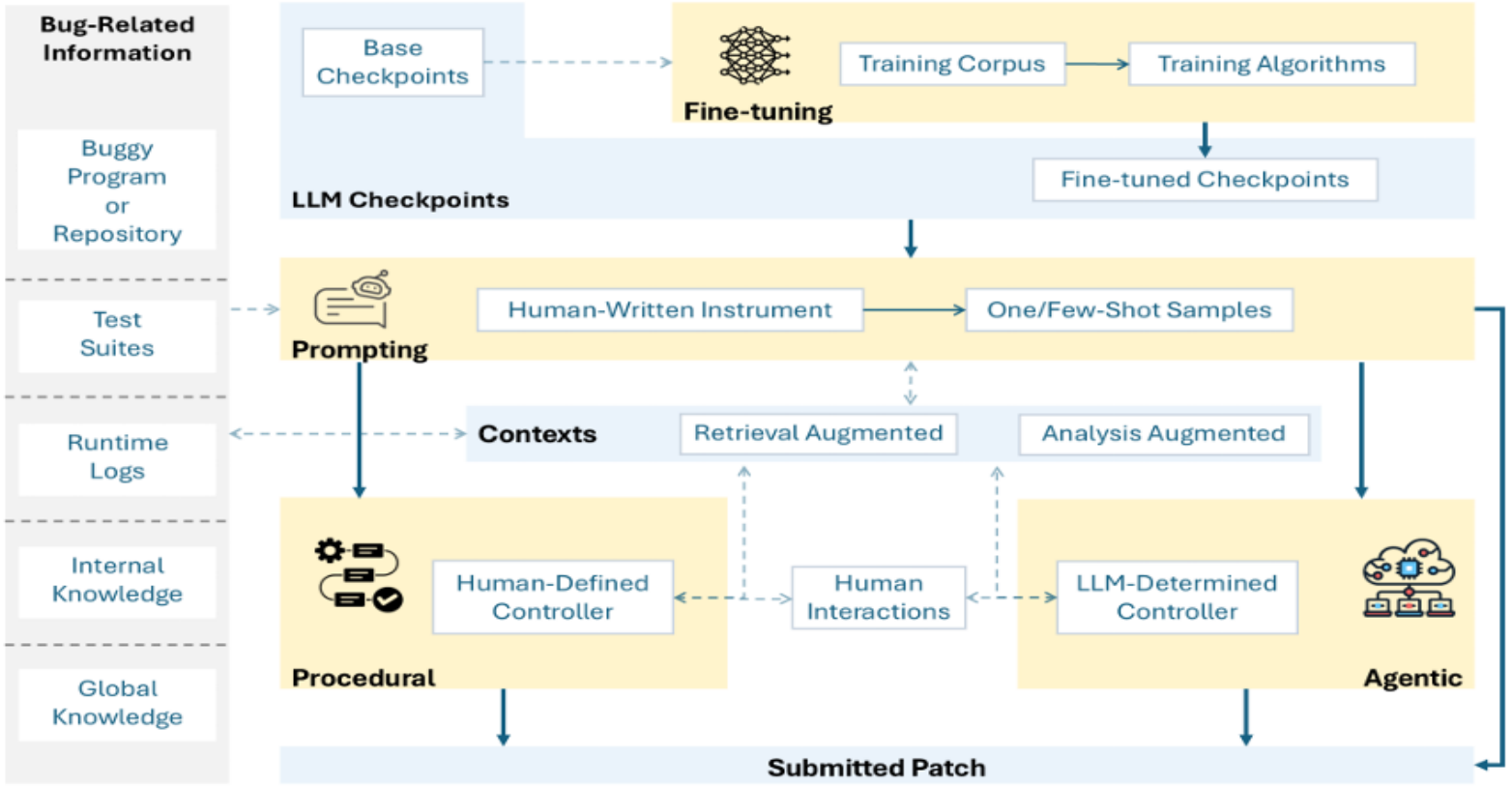
Recommendation

1. **Verify, Don't Accept**
   Human oversight of AI fixes
2. **Know the Limits**
   Concurrency I/O hallucination

Team 6: Mihiret Ali, Ting Huang, Mansi Sawant, Hamid Ahmad

Sources: Stack Overflow Dev Survey 2025 · Google DORA 2025 · GitClear 2024 (153M lines analyzed) · Chen et al. ICLR 2024

Debugging Overview: 4 LLM-based Paradigms



Team 6: Mihiret Ali, Ting Huang, Mansi Sawant, Hamid Ahmad

Suggestion for 4 LLM-based Paradigms

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Paradigm	Use when…	Avoid / be careful when…
Fine-tuning	You have enough high-quality bug-fix data and want a repair model specialized for a task, language, project type, or vulnerability class.	It can be expensive, needs training data and compute, and may reduce the model's general ability. Full fine-tuning gives strong accuracy, while Parameter-Efficient Fine-Tuning and distillation reduce cost.
Prompting	You want a fast, low-cost baseline with no model training. It is best for simpler or localized bugs, especially when the buggy code and instruction fit in one prompt.	It is sensitive to prompt wording and may be unreliable for multi-hunk, cross-file, or repository-level defects.
Procedural	You need a controlled multi-step workflow, such as generate → test → revise, or retrieval/analysis between repair attempts. This is good when you want predictable behavior and human-designed control.	It may require careful pipeline engineering, and fixed workflows may under-explore complex bugs. Procedural RAG/AAG can improve accuracy but adds preprocessing and orchestration cost.
Agentic	You need flexible repository-level repair where the system must decide which files to inspect, tools to call, patches to keep, or when to stop. This is useful for complex, multi-step repair.	It can be costly and brittle because it may involve many LLM calls, tools, and test runs. The paper notes that multi-step procedural and especially agentic workflows can have high token, tool, and latency costs.

No single best method → choice depends on task, cost, and complexity

Guideline 1
Rubber Duck Debugging: Explain-Then-Fix

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

1

Explain-Then-Fix (No tests available)

When to use: logical bugs, no unit tests, code ≤ 50 lines
When to skip: very large files, pure syntax errors (traceback already pinpoints the line)

2

Execution Trace Feedback (Tests available)

When to use: tests exist, runtime exceptions, wrong output
When to skip: no sandbox, very long logs (> 50 lines, truncate to exception + 5 lines above

Self-Debug (Chen et al., 2024)

Rubber Duck Debugging: explaining code line-by-line forces the model to surface the gap between what the code does vs. what it should do without any human feedback.

Guideline 1
Rubber Duck Debugging: Explain-Then-Fix

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Counterexample – running_average bug

```
for i, n in enumerate(numbers):
    total += n
    result.append(total / i)    # ← bug
```

Baseline: "Fix this code." → may miss edge case

Turn 1 (Explain): Model spots total/i → ZeroDivisionError on i=0, wrong denominator

Turn 2 (Fix): Model outputs total/(i+1) — correct

Counterexample –Race condition

```
# Two threads, shared counter
for _ in range(100_000):
    counter += 1    # race condition
```

Explain-then-fix fails here: the model correctly explains the code at the static level but cannot observe non-deterministic interleaving. It says: "looks correct.

Refined boundary:
Explain-Then-Fix works for **static** bugs.
For concurrency / I/O side-effects → use Execution Trace Feedback or human review.

Guideline 1
Rubber Duck Debugging: Explain-Then-Fix

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

explain_then_fix.py × Welcome

Users > mihiret > GSE-blokus-engine- > explain_then_fix.py > average

```python
def average(numbers):
    if not numbers:
        raise ValueError("Cannot compute average of empty list")
    return sum(numbers) / len(numbers)
```

Line-by-Line Explanation

Line 1: `def average(numbers):`
Defines a function named `average` that takes one parameter `numbers` (expected to be a list/sequence of numbers).

Line 2: `return sum(numbers) / len(numbers)`
Returns the arithmetic mean by dividing the sum of all numbers by the count of numbers in the list.

Correctness Assessment

❌ **NO, this function does NOT correctly handle all possible inputs.**

Issues:

1. **Empty list (CRITICAL BUG):** If `numbers` is an empty list, the function raises `ZeroDivisionError` because `len(numbers)` equals 0. Division by zero is undefined.

```
average([])  # ❌ ZeroDivisionError: d
```

Based on your explanation above, fix the function so it handles the empty list case.

Updated average function to handle empty list case

Updated explain_then_fix.py +2 -0

Done! The function now includes a check for empty lists. If an empty list is passed, it raises a `ValueError` with a descriptive message instead of crashing with a `ZeroDivisionError`.

The fixed function:

- ✅ Handles empty lists gracefully
- ✅ Computes the average correctly for non-empty lists
- ✅ Provides clear error feedback when invalid input is given

Guideline 2
AutoSD: LLM-Driven Scientific Debugging

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

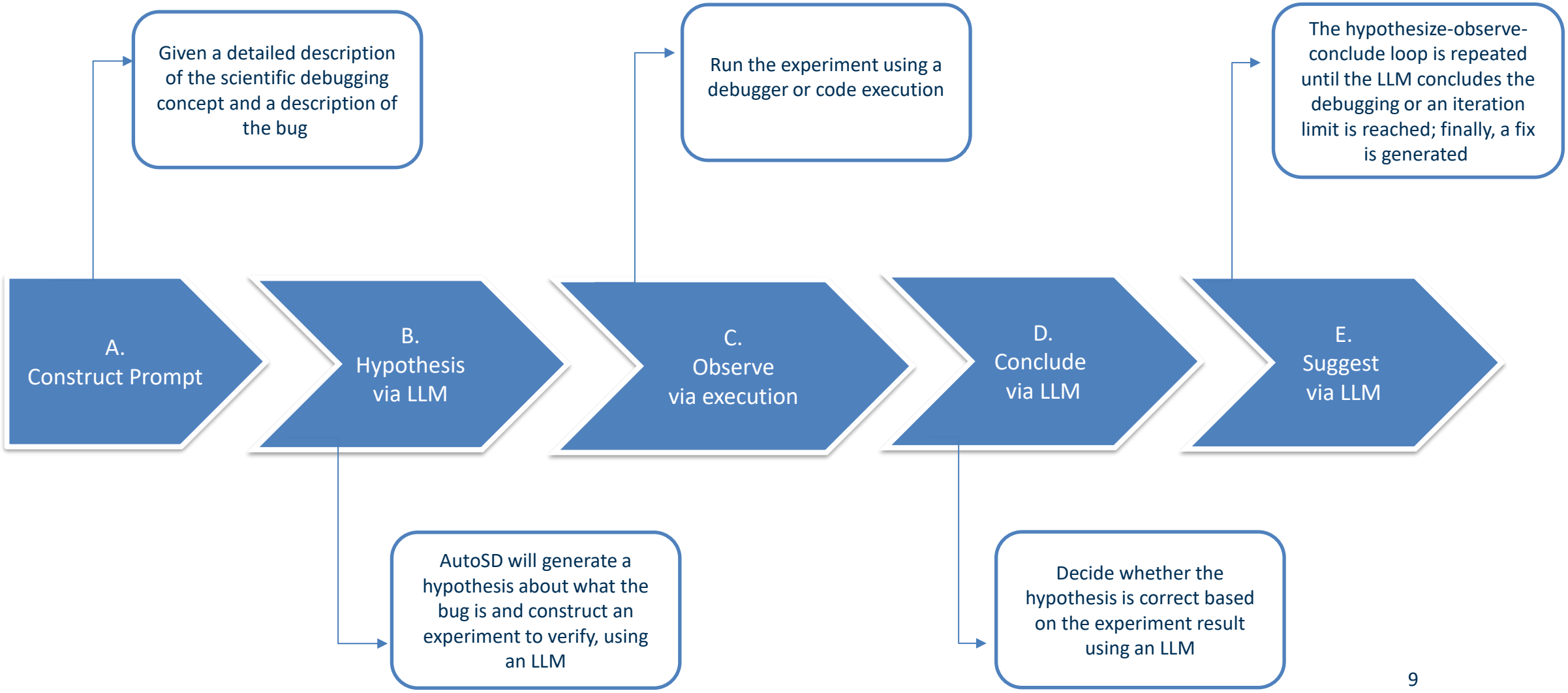
What is AutoSD?

- Automated Scientific Debugging(AutoSD) is a debugging method where **LLMs emulate how humans debug**
- Combines:
  - **LLM reasoning** + **real code execution (debugger)**
- Produces both:
  - **Bug fix**
  - **Step-by-step explanation (reasoning trace)**

AutoSD turns LLM debugging from **black-box guessing into a transparent, scientific reasoning process**.

Guideline 2
AutoSD: LLM-Driven Scientific Debugging

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics



Guideline 2
AutoSD: LLM-Driven Scientific Debugging

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Example (Incorrect Average Calculation):

```python
def calculate_average(scores):
    total = 0
    count = 0

    for s in scores:
        if s >= 0:
            total += s
        count += 1

    return total / count

# Test cases
print(calculate_average([10, 20, 30]))      # Expected: 20
print(calculate_average([10, -5, 15]))      # Expected: 12.5 (ignore negative)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\mansi> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& C:\Users\mansi\.virtualenvs\r-re
(r-reticulate) PS C:\Users\mansi> cd .\Documents\
(r-reticulate) PS C:\Users\mansi\Documents> cd .\GSE_Project\
(r-reticulate) PS C:\Users\mansi\Documents\GSE_Project> cd '.\Guideline Session\'
(r-reticulate) PS C:\Users\mansi\Documents\GSE_Project\Guideline Session> python .\example-debugging-3.py
20.0
8.333333333333334
(r-reticulate) PS C:\Users\mansi\Documents\GSE_Project\Guideline Session>
```

1.

Hypothesis

The average is incorrect because the `count` variable is incremented for **every score in the list**, including negative ones that are excluded from the `total` sum. This results in a denominator that is too large, underestimating the average.

- For `[10, 20, 30]`: Total = 60 (all positive), count = 3 → 60/3 = 20 (matches expectation).
- For `[10, -5, 15]`: Total = 25 (ignores -5), but count = 3 → 25/3 ≈ 8.33 (expected 12.5, which would be 25/2 if count only included positives).

2.

Expected Observations

1. **Correct averages for lists with no negatives:** The function works as expected. For example, `calculate_average([10, 20, 30])` outputs `20.0`, matching the expected average of 20.
2. **Incorrect (underestimated) averages for lists with negatives:** The average is lower than it should be because the denominator includes excluded scores. For example:

3.

Suggested Debugging Step

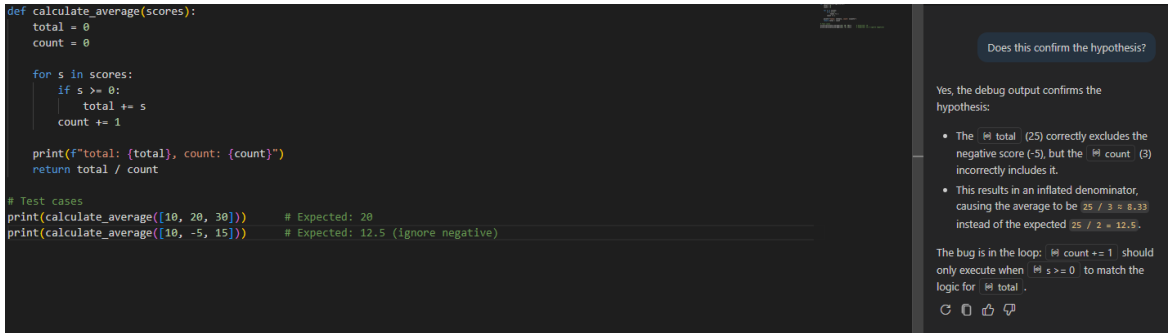
1. **Add debug prints:** Insert `print(f"total: {total}, count: {count}")` just before the `return` statement in the function. This will output the intermediate values after processing the list.
2. **Run the script:** Execute `python .\example-debugging-3.py` (from the correct directory: `C:\Users\mansi\Documents\GSE_Project\Guideline Session`).
3. **Observe the output:**

Guideline 2
AutoSD: LLM-Driven Scientific Debugging

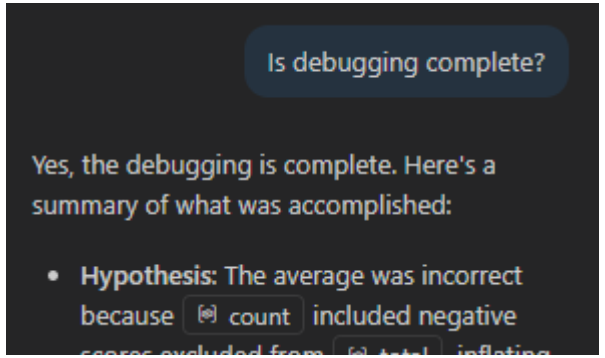
UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

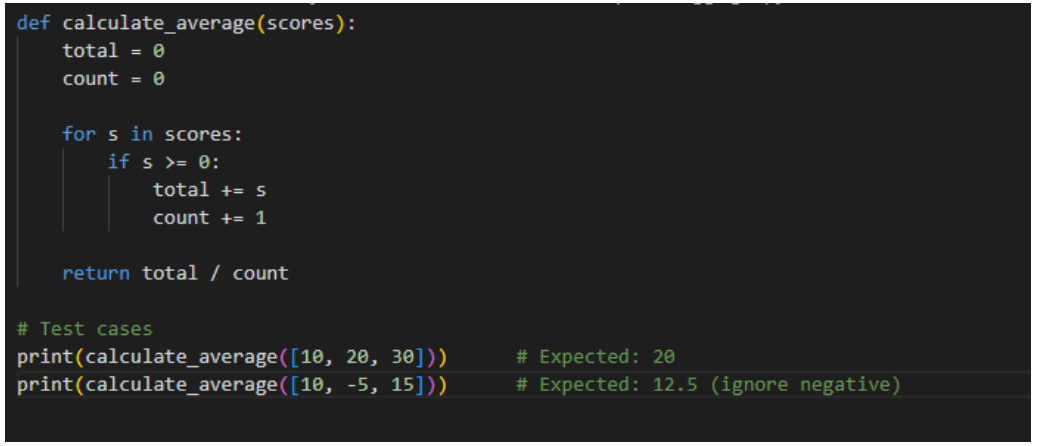
Example (Incorrect Average Calculation):

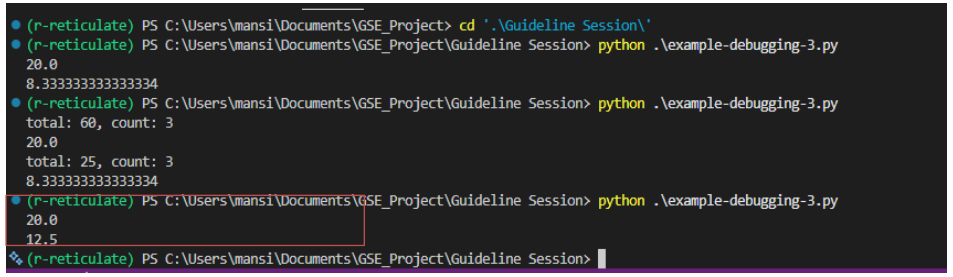
4.



5.







Guideline 3
RepairAgent Debugging

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

- What is RepairAgent Debugging?
- RepairAgent is an autonomous, LLM-based agent designed to automatically find and fix bugs in code.
- RepairAgent doesn't require fixed prompts or loops to work.
- It can handle various bug types and work with different programming languages. Iterative
- RepairAgent can gather information, apply fixes, and validate changes autonomously.
- This guideline explains how developers can integrate RepairAgent into their workflow for efficient debugging.

Guideline 3
RepairAgent Debugging

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Problem with usual Debugging

- Manual error detection and fixing takes time and expertise.
- Developers may overlook edge cases or accidentally introduce new bugs while fixing.
- Debugging can be inefficient and not scalable for large codebases.

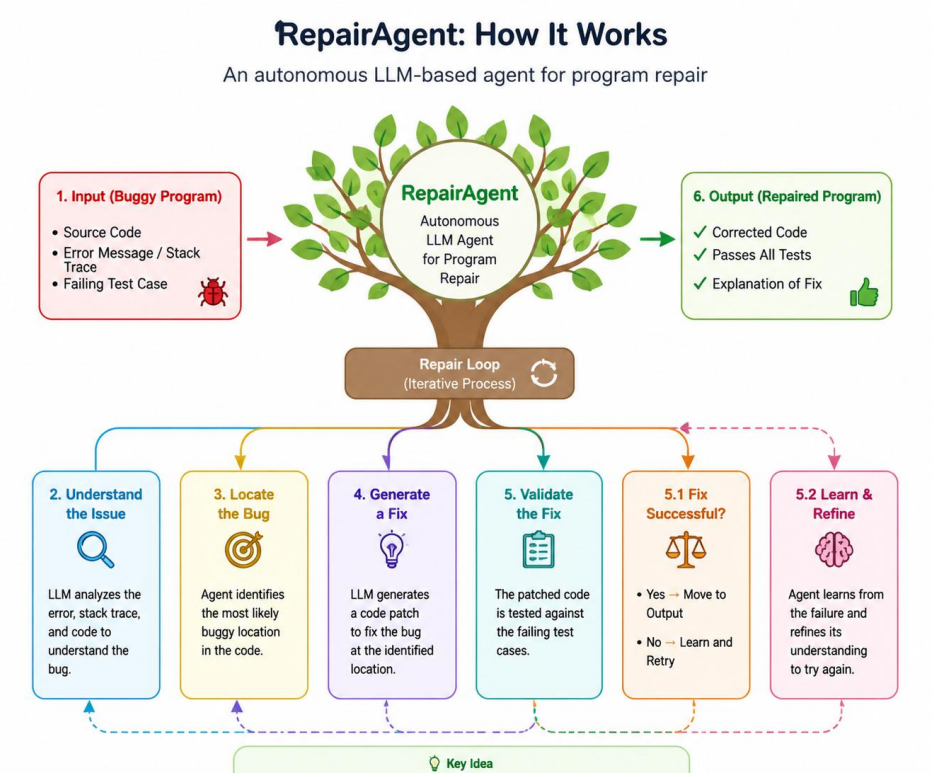
How RepairAgent Helps

- Speeds up the process of bug localization and repair.
- Fixes suggests automatically, reducing human error and saving time.
- Ensures fixes are appropriate and minimize the risk of introducing new bugs.

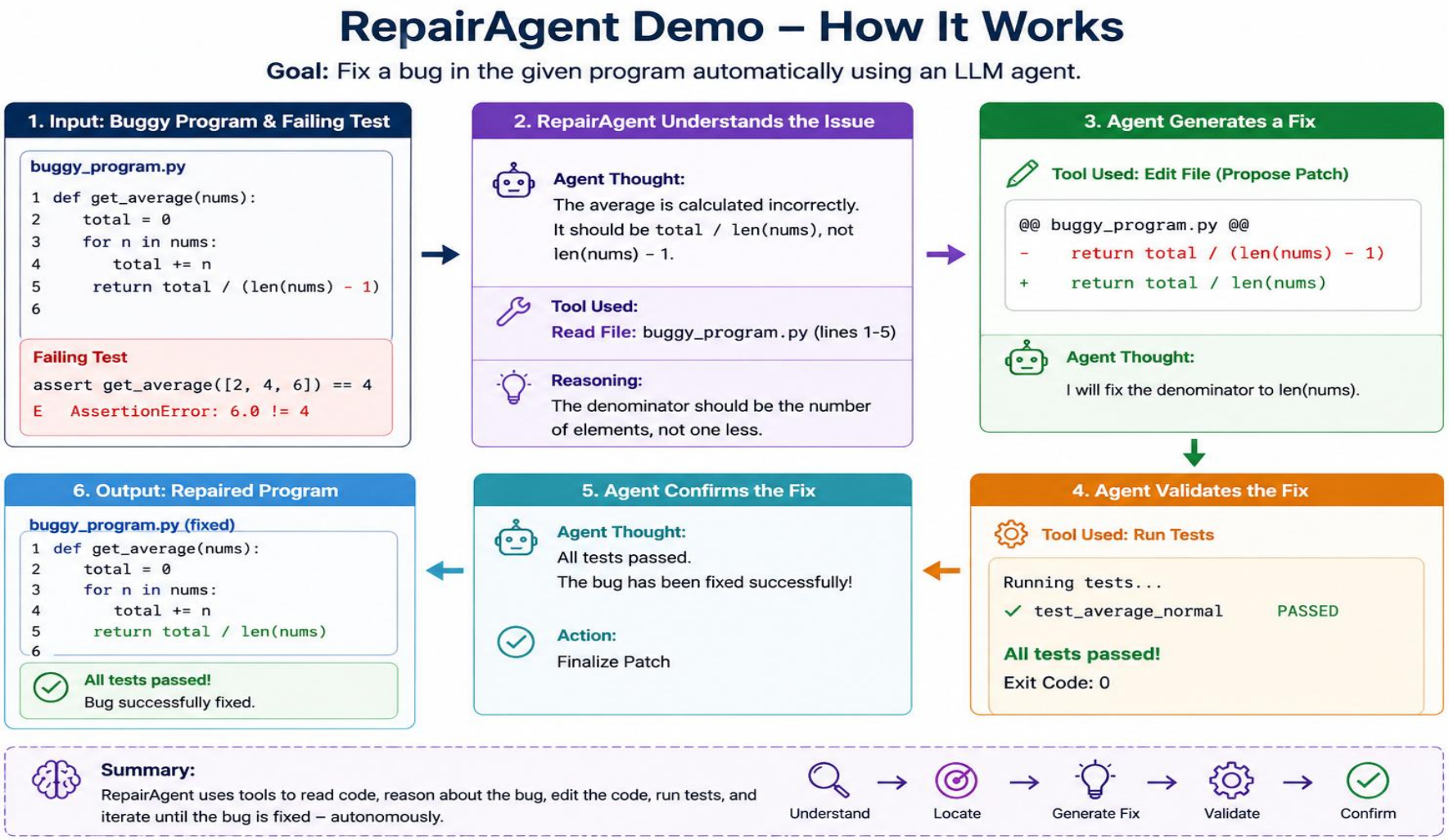
How RepairAgent Works?

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

- **Bug Detected**: RepairAgent identifies a bug in the code (e.g., **division by zero**).
- **Bug Fixed**: The agent proposes a fix, such as adding a **check to prevent division by zero**.
- **Validation**: RepairAgent ensures that the fix works **without breaking other parts** of the code.



Guideline 3


RepairAgent Demo – How It Works
Goal: Fix a bug in the given program automatically using an LLM agent.
1. Input: Buggy Program & Failing Test
buggy_program.py
1 def get_average(nums):
2 total = 0
3 for n in nums:
4 total += n
5 return total / (len(nums) - 1)
6
Failing Test
assert get_average([2, 4, 6]) == 4
E AssertionError: 6.0 != 4
2. RepairAgent Understands the Issue
Agent Thought:
The average is calculated incorrectly.
It should be total / len(nums), not len(nums) - 1.
Tool Used:
Read File: buggy_program.py (lines 1-5)
Reasoning:
The denominator should be the number of elements, not one less.
3. Agent Generates a Fix
Tool Used: Edit File (Propose Patch)
@@ buggy_program.py @@
- return total / (len(nums) - 1)
+ return total / len(nums)
Agent Thought:
I will fix the denominator to len(nums).
4. Agent Validates the Fix
Tool Used: Run Tests
Running tests...
test_average_normal PASSED
All tests passed!
Exit Code: 0
5. Agent Confirms the Fix
Agent Thought:
All tests passed.
The bug has been fixed successfully!
Action:
Finalize Patch
6. Output: Repaired Program
buggy_program.py (fixed)
1 def get_average(nums):
2 total = 0
3 for n in nums:
4 total += n
5 return total / len(nums)
6
All tests passed!
Bug successfully fixed.
Summary:
RepairAgent uses tools to read code, reason about the bug, edit the code, run tests, and iterate until the bug is fixed – autonomously.
Understand
Locate
Generate Fix
Validate
Confirm

Summary

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

- Prompt structure matters more than model choice
  - How you debug with AI matters as much as which AI you use.
- AI debugging isn't magic. 3-turn cap. Human review for concurrency, security, and architectural bugs. Vibe coding has real costs.

- Verify, dont accept
  - Always run the full test suite after an AI fix. Write a regression test for the specific bug. AI fixes pass the described symptom but may break adjacent features.
- Know the Limits
  - Explain-Then-Fix fails for concurrency, I/O side-effects, and non-deterministic bugs. When static reasoning can't see it, you need runtime evidence or human review.

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Thanks for your attention!

Literature

UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

- Chen, Xinyun, et al. "Teaching large language models to self-debug." arXiv preprint arXiv:2304.05128 (2023). https://arxiv.org/pdf/2304.05128
- Kang, S., Chen, B., Yoo, S. et al. Explainable automated debugging via large language model-driven scientific debugging. Empir Software Eng 30, 45 (2025). https://doi.org/10.1007/s10664-024-10594-x
- Yang, Boyang, et al. "A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications." arXiv preprint arXiv:2506.23749 (2025). https://arxiv.org/pdf/2506.23749
- Repairagent: An autonomous, llm-based agent for program repair." 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025. https://arxiv.org/pdf/2403.17134